Insertion Sort :

```
#include <stdio.h>

int main() {
   int numbers[100], key, i, j, n;

   printf("Enter the dimension of the array: ");
   scanf("%d", &n);

   // array input
   for(i = 0; i < n; i++) {
      printf("Enter numbers[%d]: ", i);
      scanf("%d", &numbers[i]);
   }

   // insertion sort
   for(j = 1; j < n; j++) {
      key = numbers[j];
      i = j - 1;

      while (i >= 0 && numbers[i] > key) {
         numbers[i + 1] = numbers[i];
         i = i - 1;
      }

      numbers[i + 1] = key;
   }

   // printing the sorted array
   for(i = 0; i < n; i++) {
      printf("%d ", numbers[i]);
   }

   return 0;
}
```

Binary Search

```
#include <stdio.h>
void Binarysearch(int arr[], int low, int high, int key)
{
   int mid;
   while(low <= high)
   {
      mid = (high + low) / 2;

      if(arr[mid] == key)
      {
         printf("\n Key is found at index %d", mid);
         break;
      }
      else if(arr[mid] > key)
         high = mid - 1;
      else
         low = mid + 1;
   }
   if(low > high)
      printf("\n Key not found...!!!");
}
int main()
{
   int a[100], key, n, i;
   printf("\n Enter the number of elements in the array: ");
   scanf("%d", &n);
   printf("\n Enter the elements in the array in sorted manner: ");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
   printf("\n Enter the key to be searched: ");
   scanf("%d", &key);

   Binarysearch(a, 0, n, key);

   return 0;
}
```

Maximum and Minimum Element in C

```
#include <stdio.h>
int max, min;
int a[100];
void maxmin(int i, int j)
{
   int max1, min1, mid;
   if(i == j)
   {
      max = min = a[i];
   }
   else
   {
      if(i == j - 1)
      {
         if(a[i] < a[j])
         {
            max = a[j];
            min = a[i];
         }
         else
         {
            max = a[i];
            min = a[j];
         }
      }
      else
      {
         mid = (i + j) / 2;
         maxmin(i, mid);
         max1 = max;
         min1 = min;
         maxmin(mid + 1, j);
         if(max < max1)
            max = max1;
         if(min > min1)
            min = min1;
      }
   }
}
```

```
int main()
{
   int i, num;

   printf("\nEnter the total number of numbers: ");
   scanf("%d", &num);

   printf("Enter the numbers:\n");

   for(i = 1; i <= num; i++)
      scanf("%d", &a[i]);

   max = a[0];
   min = a[0];

   maxmin(1, num);

   printf("Minimum element in an array: %d\n", min);
   printf("Maximum element in an array: %d\n", max);

   return 0;
}
```

Longest Common Subsequence

```
#include <stdio.h>
#include <string.h>

int i, j, m, n, LCS_table[20][20];
char S1[20] = "ACADB", S2[20] = "CBDA", b[20][20];

void lcsAlgo() {
   m = strlen(S1);
   n = strlen(S2);

   // Filling 0's in the matrix
   for(i = 0; i <= m; i++)
      LCS_table[i][0] = 0;

   for(i = 0; i <= n; i++)
      LCS_table[0][i] = 0;

   // Building the matrix in bottom-up way
   for(i = 1; i <= m; i++) {
      for(j = 1; j <= n; j++) {
         if(S1[i - 1] == S2[j - 1]) {
            LCS_table[i][j] = LCS_table[i - 1][j - 1] + 1;
         }
         else if(LCS_table[i - 1][j] >= LCS_table[i][j - 1]) {
            LCS_table[i][j] = LCS_table[i - 1][j];
         }
         else {
            LCS_table[i][j] = LCS_table[i][j - 1];
         }
      }
   }

   int index = LCS_table[m][n];
   char lcsAlgo[index + 1];
   lcsAlgo[index] = '\0';

   int i = m, j = n;

   while(i > 0 && j > 0) {
```

```
      if(S1[i - 1] == S2[j - 1]) {
         lcsAlgo[index - 1] = S1[i - 1];
         i--;
         j--;
         index--;
      }
      else if(LCS_table[i - 1][j] > LCS_table[i][j - 1])
         i--;
      else
         j--;
   }

   // Printing the subsequences
   printf("S1 : %s \nS2 : %s \n", S1, S2);
   printf("LCS: %s", lcsAlgo);
}

int main() {
   lcsAlgo();
   printf("\n");
   return 0;
}
```

Floyd Warshall Algorithm

```
#include <stdio.h>

int i, j, k, n, dist[10][10];

void floydwarshall()
{
   for(k = 0; k < n; k++)
      for(i = 0; i < n; i++)
         for(j = 0; j < n; j++)
            if(dist[i][k] + dist[k][j] < dist[i][j])
               dist[i][j] = dist[i][k] + dist[k][j];
}

int main()
{
   int i, j;

   printf("Enter no of vertices: ");
   scanf("%d", &n);

   printf("\n");

   for(i = 0; i < n; i++)
      for(j = 0; j < n; j++)
      {
         printf("dist[%d][%d]: ", i, j);
         scanf("%d", &dist[i][j]);
      }
   floydwarshall();
   printf("\n\nShortest distances between every pair of vertices\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
         printf("%d\t", dist[i][j]);
      printf("\n");
   }
   return 0;
}
```

Fractional Knapsack

```
#include <stdio.h>

int main()
{
   int n, w[50], p[50], obj[50], i, j, temp1;
   float xr, x[50], total_profit = 0, total_weight = 0;
   float ratio[50], u, m, temp;

   printf("\nEnter number of items : ");
   scanf("%d", &n);

   printf("Enter capacity of knapsack : ");
   scanf("%f", &m);

   u = m;

   for(i = 0; i < n; i++)
   {
      x[i] = 0;
      obj[i] = i + 1;
   }

   printf("\nEnter the weights of each object\n");
   for(i = 0; i < n; i++)
   {
      printf("Weights of object %d = ", i + 1);
      scanf("%d", &w[i]);
   }

   printf("\nEnter the profit of each object\n");
   for(i = 0; i < n; i++)
   {
      printf("Profit of object %d = ", i + 1);
      scanf("%d", &p[i]);
   }

   for(i = 0; i < n; i++)
   {
```

```
   ratio[i] = (float)p[i] / (float)w[i];
}

// Sorting according to profit-weight ratio
for(i = 0; i < n; i++)
{
   for(j = 0; j < n - 1; j++)
   {
      if(ratio[j] < ratio[j + 1])
      {
         temp = ratio[j];
         ratio[j] = ratio[j + 1];
         ratio[j + 1] = temp;

         temp1 = w[j];
         w[j] = w[j + 1];
         w[j + 1] = temp1;

         temp1 = p[j];
         p[j] = p[j + 1];
         p[j + 1] = temp1;

         temp1 = obj[j];
         obj[j] = obj[j + 1];
         obj[j + 1] = temp1;
      }
   }
}

printf("\nTable after sorting according to profit-weight ratio :\n");

printf("\nObject:\t\t");
for(i = 0; i < n; i++)
   printf("%d\t", obj[i]);

printf("\nProfit:\t\t");
for(i = 0; i < n; i++)
   printf("%d\t", p[i]);

printf("\nWeights:\t");
```

```
for(i = 0; i < n; i++)
   printf("%d\t", w[i]);

printf("\nRatio:\t\t");
for(i = 0; i < n; i++)
   printf("%f\t", ratio[i]);

printf("\n");

// Selecting objects
for(i = 0; i < n; i++)
{
   if(w[i] <= u)
   {
      x[i] = 1;
      u = u - w[i];
   }
   else if(w[i] > u)
   {
      break;
   }
}

if(i < n)
{
   xr = (float)u / (float)w[i];
   x[i] = xr;
}

printf("\nX=[");
for(i = 0; i < n; i++)
{
   printf("%.3f, ", x[i]);
}
printf("]");

for(i = 0; i < n; i++)
{
   total_profit += x[i] * p[i];
   total_weight += x[i] * w[i];
```

```
    }

    printf("\n\nTotal weight=%.2f", total_weight);
    printf("\nTotal profit=%.2f\n", total_profit);

    return 0;
}
```